Queue Model Implementation
Practical Session Report

Universitat Politécinca de Catalunya - FIB

Prepared by:
Joel González Jiménez

Date: June 18, 2026
Course: SMDE

1 Executive Summary

As blockchain networks experience exponential growth in transaction volumes, efficiency and processing speed have become critical challenges for network viability. This report analyzes the performance bottlenecks within the complete block lifecycle, from the initial mining phase to final verification, using an event scheduling simulation engine. Utilizing a fractional factorial Design of Experiments (DoE), we systematically evaluate key operational factors to identify the optimal parameter configuration that maximizes block validation throughput.

2 System Description and Introduction

The system is a simplification of a blockchain network, instead of transactions we consider that blocks arrive to the network, the nodes mine the blocks, then they broadcast the block mined to peers, finally the all the block data is validated and consolidated in the chain.

The simulated system contains three queues sequentially connected ($Queue1 \rightarrow Queue2 \rightarrow Queue3$) each one with an associated task with predefined service time (10, 6, 7) respectively, the amount of servers in each queue are (c, 1, 1) being 'c' a parameter affected by a factor.

BPMN Mapping

- **Entry Events:** Represent the arrival of pre-compiled blocks containing transaction batches. The inter-arrival times are modeled using an exponential distribution (M).
- **Lanes:** Each lane maps directly an isolated queue within the system.
- **Service Tasks:** The pipeline comprises three distinct execution tasks. For architectural simplicity, the service times for all tasks follow continuous uniform distributions (U), with each task maintaining its own dedicated waiting queue:
    1. **Task 1 — Mining (M/U/c):** block mining and preparation.
    2. **Task 2 — Broadcasting (U/U/1):** Network propagation of the mined block to peer nodes.
    3. **Task 3 — Validation (U/U/1):** Cryptographic verification of the block and chain consolidation.

- **Routing Dynamics:** To facilitate the validation, the structure is configured as a sequential, multi-stage queueing network.

Simulation Focus

The primary simulation focus centers on evaluating the system throughput, defined as the rate of validated blocks successfully appended to the blockchain upon completing the three-stage processing pipeline.

3 Problem Description

Factors in the system

The simulated system is affected by four quantitative factors, table 1 shows which aspects of the simulation are affected by each factor of the simulation model.

Name	Affected Element/s	Affected Parameter/s	Equation/s
Block Difficulty	Task1	service time	(1)
Peers	Task1 & Task2	servers & service time	(2) & (3)
Block Data	Task3	service time	(4)
Interarrival Rate	Entry Events	arrival rate	(5)

Table 1: Factors impact in the system.

The equations of the impact of factors to model parameters are the following:

$$\text{Task1 service time} = \text{initial service time} * (\text{Block Difficulty}/10) \tag{1}$$

$$\text{Task1 servers} = \text{initial servers} * \text{Peers} \tag{2}$$

$$\text{Task2 service time} = \text{initial service time} * (\text{Peers}/10) \tag{3}$$

$$\text{Task3 service time} = \text{initial service time} * (\text{Block Data}/20) \tag{4}$$

$$\text{arrival rate}(\lambda) = \frac{1}{\text{Interarrival Rate}} \tag{5}$$

Problem thought before simulation

The objective in this simulation is to maximize the throughput of blocks validation, but in this configuration the mining process could be a huge bottleneck if both factors 'Peers' and *Block Difficulty* are unfavorable. While a higher value of *Peers* is better for lowering the number of elements in Queue1, it is worse for Queue2, so a problem will be finding a good balance.

Problem found after simulation

After the execution we found that *Peers* factor had a significant impact on the block validation throughput, but the positive effect on Queue1 congestion was much important than the negative one to Queue2, so the optimum value of *Peers* would be above the value in the middle. Also we found that factor *Block Difficulty* and *Interarrival Rate* had big impact and they are a problem if their value are high, lower is better.

3.1 Hypotheses Framework

To complete the system bounds, the structural behavior, parameters, and limitations are framed under the explicit identifiers listed below:

SI (Simplifying)

ST (Structural)

SY (Systemic)

SI_01: The cryptographic difficulty across all arriving blocks is assumed to be homogeneous and static.

SI_02: Network transit and message propagation delays between the physical queue stages are negligible ($t = 0$), meaning sequence flow movements are instantaneous.

SI_03: The blockchain network is modeled as a single, closed linear pipeline; structural complexities such as network splits, malicious nodes, or block forks are omitted.

ST_01: Every individual processing stage maintains a strict, non-preemptive First-In-First-Out ($FIFO$).

ST_02: The storage capacity (N) for all system queues is assumed to be infinite, ensuring that blocks are never dropped due to overflow constraints.

ST_03: When an arriving block finds an available processing server, it bypasses queue buffering immediately, experiencing a queue waiting time of exactly zero ($W_q = 0$).

ST_04: The calculation of the global average waiting time explicitly includes immediate-service transactions ($W_q = 0$).

SY_01: The block entry stream behaves as a memoryless Poisson process, where inter-arrival times are drawn from a synthetic continuous exponential distribution (M).

SY_02: Service times are modeled using continuous uniform distributions (U), where the explicit boundaries $[a, b]$.

4 Model Specification

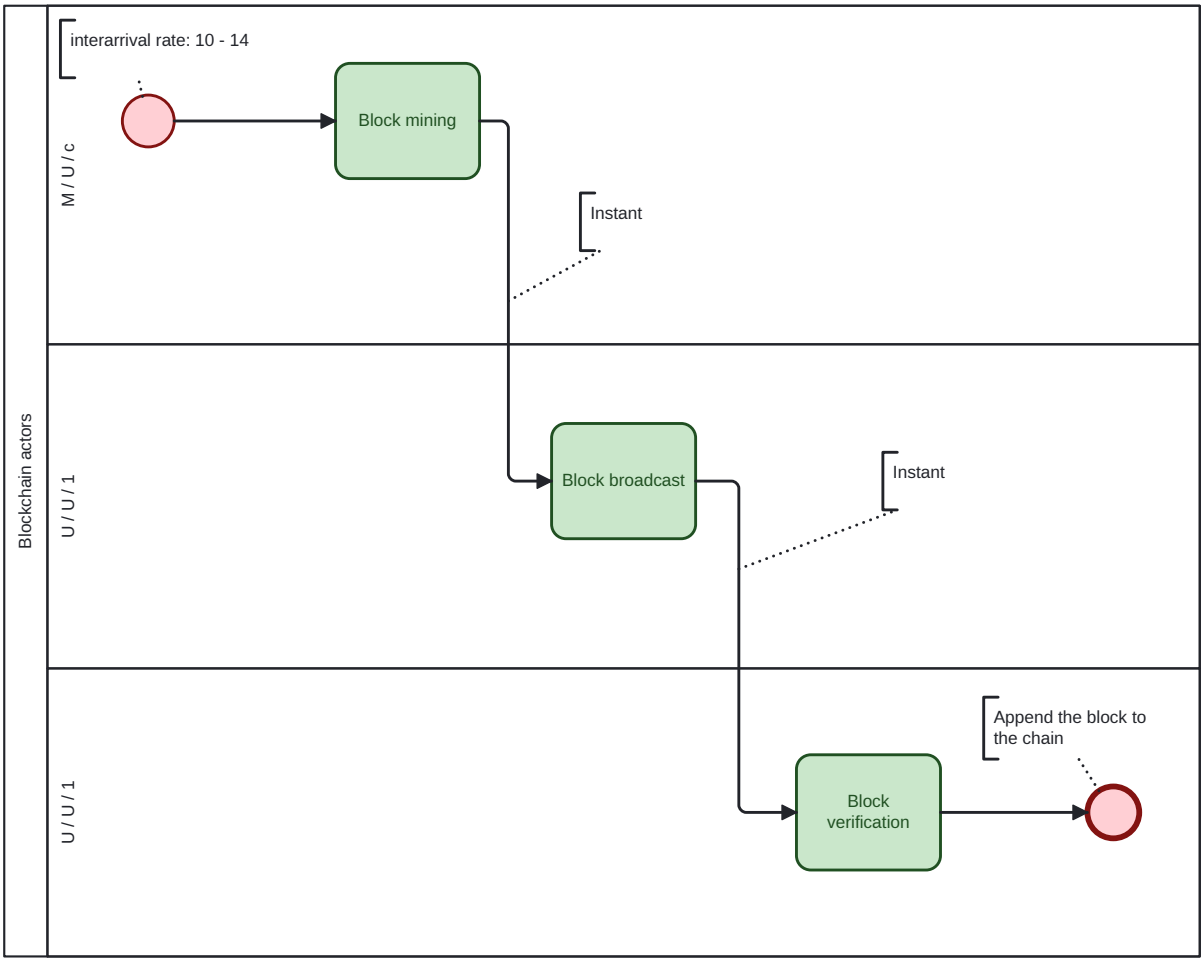

Figure 1: BPMN flow specification representing the sequential block validation pipeline ($M/U/c \rightarrow U/U/1 \rightarrow U/U/1$).

5 Coding

The implementation is a event scheduling simulation engine written in Python. The simulator operates chronologically using event scheduling mechanics rather than fixed steps. The queue topology and factors information is provided by a .yaml where it can defined any kind of topology. A definition example can be found in the appendix B, in order to define the topology always start with a virtual queue 'Queue0' that introduces to the first real queue .The program has the following five classes:

Engine

Responsibilities of the class:

- Manages the events coming from the future event list, `self.fec`, the events in the list are ordered by the move time. An event can be either a 'ARRIVAL' or a 'DEPARTURE'.

- Manages the simulated time through a clock `self.clock` and the deadline of the simulation `self.deadline`.
- It controls the arrivals given a mean interarrival rate within an exponential distribution.
- Has the routing configuration between queues `self.topology` and is responsible of attaching events to queues `self.queues`, in other words it manages the entire event life cycle.

```
def __init__(self, deadline: float, seed: int, routing_config: Any):
    self.clock: float = 0 # Current simulation logical time
    self.deadline: float = deadline # Global simulation ending
    self.fec: list[Event] = [] # Future Event Calendar (min-heap priority queue)
    self.rng = RNG(seed)

    # Extract operational input factor parameters from configuration
    self.mean_interarrival: float = routing_config['factors']['interarrival_rate']
    self.topology: Dict[str, Any] = routing_config['topology']

    # Initialize the network nodes (KQueue instances) dynamically
    self.queues: Dict[str, KQueue] = self.queues_creation(routing_config['components'], routing_config['factors'], self.rng)

    # Statistics
    self.blocks_validated: int = 0

```

Listing 1: Engine constructor

The code in appendix A shows the loop that controls the life cycle of events. It is self explanatory with comments.

KQueue

The `KQueue` class encapsulates the behavior of an individual queue within the simulation. Its primary structural components and functional elements are detailed below:

- **`self.name`:** Each instance maintains a distinct `name` property used for routing.
- **`self.queue`:** Data structure that safely holds arriving entities (blocks) in memory while they wait for resource allocation.
- **`self.servers`:** Consists of a collection of homogeneous parallel servers with uniform service times.
- **`self.wait_times` & `self.count_history`:** Used to provide the statistical information used for model validation and to extract the metric we want to measure.

```
def __init__(self, name: str, attributes: Any, factors: Any, rng: RNG):
    self.name: str = name
    self.queue: deque[tuple[float, Event]] = deque()
    self.servers: list[Server] = self.servers_creation(attributes, factors, rng)

    # Statistics
    self.wait_times: list[float] = [] # Stores individual queuing delays (W_q)
    self.count_history: list[tuple[float, int]] = [] # Tracks state trace steps as (timestamp, queue_size)

```

Listing 2: KQueue constructor

Server

Its only purpose to process the events to do a certain job that lasts as much as the service time indicates. It can only process one event at a time.

Relevant elements:

- **`self.status`.** It is a binary value that can be either free or busy.
- **`self.service_time`.** Indicates the duration of the job.

```
def __init__(self, id: int, type: int, attributes: Any, factors: Any, rng: RNG):
    self.id: int = id
    self.status = ServerState.FREE

    # Calculate the service time modified by active experimental factors
    self.service_time: float = self.calculate_servicetime(type, attributes, factors)

    # Define uniform deviation boundaries (bounds are +/- 10% of the calculated base service time)
    self.service_deviation: float = self.service_time/10
    self.rng = rng

```

Listing 3: Server constructor

Event

This class only contains the necessary information for the engine to handle the events.

Relevants elements:

- **self.move_time.** The time has to be performed the next move.
- **self.type.** Indicates wether it is an arrival or a departure event.
- **self.queue_name.** Indicates the last/actual queue the event was.
- **self.server_id.** Indicates the last/actual server the event passed.

```
def __init__(self, id: int, moveTime: float, type: EventType,
    queue_name: str, server_id: int):
     self.id: int = id
     self.move_time: float = move_time
     self.type: EventType = type # Event classification (e.g.,
    ARRIVAL, DEPARTURE)
     self.queue_name: str = queue_name
     self.server_id: int = server_id # Identifier of the active
    server handling the process (-1 if none)

```

Listing 4: Event constructor

RNG

This class uses congruential generator in order to provide the generation of random numbers.

```
    class RNG:
        # Constants
        M: int = 2**31
        A: int = 1103515245
        C: int = 12345

        def __init__(self, seed: int):
            self.seed: int = seed

        def generate_number(self) -> float:
            self.seed = (self.A * self.seed + self.C) % self.M
            return float(self.seed / self.M)

```

Listing 5: RNG class implementation

6 Data

It has not been used data for the simulations.

7 Definition of the Experimental Framework

To optimize experimental efficiency, this study utilizes a 2^{4-1} fractional factorial design. This methodology reduces the required experimental overhead by 50% while maintaining

the statistical power necessary to evaluate main factor effects and two-way interactions. The primary response variable is block validation throughput. To standardize measurements across all experiments and independent replications, each simulation run is bound to a fixed execution window of 10,000 time units. Consequently, the optimization objective is to identify the parameter configurations that maximize the cumulative number of validated blocks within this uniform temporal baseline.

The parametrization of two levels for each factor is shown in table 2.

A (Block Difficulty)	**B (Peers)**	**C (Block Data)**	**D (Interarrival Rate)**
+8	+4	+8	+13
-2	-1	-2	-11

Table 2: Two level factors values.

To determine the number of replications needed we performed a ten pilot replications executions and this were the results:

$$X = \{750, 756, 771, 781, 746, 783, 790, 743, 739, 751\}$$

$$\bar{X} = 761.0 \tag{6}$$

$$S = 18.196 \tag{7}$$

The initial half-range (h) of the 95% confidence interval is determined by:

$$h = t_{\alpha/2,n-1} \cdot \frac{S}{\sqrt{n}} = 2.262 \cdot \frac{18.196}{\sqrt{10}} = 13.016 \tag{8}$$

Setting a strict maximum allowable error threshold of 2% ($\gamma = 0.02$), the desired target half-range (h^*) is defined as:

$$h^* = 0.02 \cdot \bar{X} = 0.02 \cdot 761.0 = 15.220 \tag{9}$$

Applying the structural replication approximation model, the total required sample size (n^*) evaluates to:

$$n^* = n \left(\frac{h}{h^*} \right)^2 = 10 \left(\frac{13.016}{15.220} \right)^2 \approx 7.31 \tag{10}$$

Mathematically is required ($n \approx 8$) replications. In table 3 can be seen the results with its corresponding parametrization

A	B	C	D	Results
(-2)	(-1)	(-2)	(-11)	824.625
(+8)	(-1)	(-2)	(+13)	553.875
(-2)	(+4)	(-2)	(+13)	779.625
(+8)	(+4)	(-2)	(-11)	906.750
(-2)	(-1)	(+8)	(+13)	765.000
(+8)	(-1)	(+8)	(-11)	554.250
(-2)	(+4)	(+8)	(-11)	898.125
(+8)	(+4)	(+8)	(+13)	765.000

Table 3: Fractional factorial tables with results.

Next, the Yates algorithm is applied to the fractional factorial design data to calculate and interpret the system's main effects and interactions, table 4.

Results	(1)	(2)	(3)	Div.	Effect ID	Factor/Effect
824.625	1,378.500	3,064.875	6,047.250	8	755.906	mean
553.875	1,686.375	2,982.375	-487.500	4	-121.875	A
779.625	1,319.250	-143.625	651.750	4	162.938	B
906.750	1,663.125	-343.875	475.500	4	118.875	AB
765.000	-270.750	307.875	-82.500	4	-20.625	C
554.251	127.125	343.875	-200.250	4	-50.063	AC
898.125	-210.750	397.875	36.000	4	9.000	BC
765.000	-133.125	77.625	-320.250	4	-80.063	D (ABC)

Table 4: Experimental results and effects.

Statistical Interpretation of Experimental Effects

From the results, it can be seen that Factor B (*Peers*) shows the most positive main effect (+162.938), demonstrating that expanding peer availability significantly eases processing constraints within the defined experimental interval. Conversely, Factor A (*Block Difficulty*) and Factor D (*Block Interarrival Rate*) present severe negative main effects of -121.875 and -80.063, respectively. High levels in either parameter trigger severe queuing delays that negatively affect system throughput.

The analysis reveals a massive positive interaction element of +118.875, mathematically attributed to the aliased pair $AB = CD$. Following the sparsity-of-effects principle, this is highly likely to be driven by the AB (*Block Difficulty* $\times$ *Peers*) interaction, given the magnitude of its corresponding main effects. This establishes that the throughput degradation caused by elevated cryptographic difficulty is substantially mitigated when the network operates with a higher number of peers

Additionally, a moderate negative interaction of -50.063 is observed for the aliased pair $AC = BD$. In contrast, the minor significance of Factor C (-20.625) and the negligible interaction $BC = AD$ (+9.000) indicate that block data alterations alone do not represent a primary bottleneck within this specific system architecture.

8 Validation

In this project have performed one validation to the model and another one to the RNG.

RNG validation

For the RNG validation we performed 5 different tests with a sample size of 10000 numbers generated. This are the results of the tests performed:

- **Kolmogorov-Smirnov:** 0.375 p-value
- **Cramer Von Mises:** 0.466 p-value
- **ChiSquare:** 0.448 p-value
- **Goodness of Fit:** 0.412 p-value
- **ttest:** 0.383 p-value

This results tells us that all tests failed to reject the null hypothesis of uniformity, so the RNG seems to provide random numbers correctly. The code is provided in Appendix C.

Model validation

The model is validated through a GPSS implementation comparing two metrics:

- **AVG queue length**
- **AVG wait time in the queue**

In order to perform the validation correctly we defined an scenario with the following values for each factor:

- **Block Difficulty:** 7
- **Peers:** 2
- **Block Data:** 6
- **Interarrival Rate:** 12

Once we run both implementations with eight replicas we compare the metrics between same queues using ttest and we got the following results:

- **Queue1 — avg queue length:** 0.286 p-value
- **Queue1 — avg wait time:** 0.295 p-value
- **Queue2 — avg queue length:** 0.531 p-value
- **Queue2 — avg wait time:** 0.384 p-value
- **Queue3 — avg queue length:** 0.124 p-value
- **Queue3 — avg wait time:** 0.131 p-value

Since the results of all p-values > 0.05, we fail to reject the hypothesis of similarity. So the simulation engine implemented seems to be adequate for event scheduling. The results can be checked by executing `python3 main.py` with the same parameters and then `python3 model_validation`.

9 Results and Conclusions

This project successfully developed a event scheduling, event-driven simulation engine to analyze operational bottlenecks within a simplified blockchain architecture. By leveraging a 2^{4-1} fractional factorial Design of Experiments (DoE) and the Yates algorithm, factor main effects and interactions were quantified.The findings demonstrate that *Block Difficulty* (A), *Peers* (B), and *Block Interarrival Rate* (D) are the critical determinants of system performance, while *Block Data* (C) is negligible. To maximize block validation throughput, parameters should be adjusted according to their statistical weight: maximize peer availability (↑B), minimize cryptographic difficulty (↓A), and reduce traffic congestion (↓D). Ultimately, scaling network peer infrastructure (↑B) provides the best mitigation against system delays and yields the most noticeable throughput improvement.

A Engine Code

```
def run(self):
    # Generation of the first event
    nextId: int = 0
    self.generator(nextId, self.calculate_next_arrival(),
   EventType.ARRIVAL, self.QUEUE0, -1)

    while self.fec:
    # Teleport to the next event
    event: Event = heapq.heappop(self.fec)
    self.clock = event.move_time

    # stops immediatly the simulation if deadline is reached
    if self.clock > self.deadline:
        break

    if event.type == EventType.ARRIVAL:
        # routes to the first queue and generates new arrival
        self.process_arrival(event, nextId)
        nextId += 1

    elif event.type == EventType.DEPARTURE:
        # routes to the next queue and let the next event in the
   queue enter the server
        self.process_departure(event)

```

Listing 6: Loop that manages event life cycle

B Topology and factors definition

```
factors:
    block_difficulty: 7 # 0-10
    peers: 2 # 1-5
    block_data: 6 # 0-10
    block_interarrival_rate: 12 # 10-14

components:
    Queue1:
        mean: 10
        servers: 1

    Queue2:
        mean: 6
        servers: 1

    Queue3:
        mean: 7
        servers: 1
```

```

topology:
    Queue0:
        next:  Queue1

    Queue1:
        next:  Queue2

    Queue2:
        next:  Queue3

    Queue3:
        next:  END

```

Listing 7: Simualtion factor and topology definition

C RNG Validation Engine (R Test Output)

```python
def generate_lcg(n: int, seed: int, m: int, a: int, c: int) -> np
    .ndarray:
    rng_values = np.zeros(n)
    current = seed

    for i in range(n):
        current = (a * current + c) % m
        rng_values[i] = float(current / m)

    return rng_values

```

Listing 8: Congruential generator

```python
M: int = 2**31
A: int = 1103515245
C: int = 12345
SEED: int = 1
# Generate 10,000 numbers
n_samples: int = 10000

numbers_custom = generate_lcg(n_samples, SEED, M, A, C)

# Kolmogorov-Smirnov Test for Uniformity
ks_result = stats.kstest(numbers_custom, 'uniform')

# Cramer Von Misesfor Uniformity
cramer_result = stats.cramervonmises(numbers_custom, 'uniform')

# ChiSquare
counts, _ = np.histogram(numbers_custom, bins=np.linspace(0, 1,
    11))
expected = [len(numbers_custom) / 10] * 10
chi_result = stats.chisquare(f_obs=counts, f_exp=expected)

# Goodness of Fit using Filliben's test
goodness_fit_result = stats.goodness_of_fit(stats.uniform,
    numbers_custom, statistic='filliben')

# ttest
ttest_result = stats.ttest_1samp(numbers_custom, popmean=0.5)

# Interpret the results
interpret(ks_result.pvalue, "Kolmogorov-Smirnov (Uniformity)")
interpret(cramer_result.pvalue, "Cramer Von Mises (Uniformity)")
interpret(chi_result.pvalue, "ChiSquare (Uniformity)")
interpret(goodness_fit_result.pvalue, "Goodness of Fit (
    Uniformity)")
interpret(ttest_result.pvalue, "ttest")

```

Listing 9: RNG validation